Deney Yaprağı 1 – Ek: Hatalı Girdileri Tanıyan Sayı Sistemi Dönüştürücüsü

Kod: `deney1_donusturucu.c` (C99). Derleme ve çalıştırma: `gcc -std=c99 -Wall deney1_donusturucu.c -o deney1 && ./deney1`. Kodun tamamı ve program çıktısı bu ekin sonundadır.

1. Algoritmalar

Kaynak tabanı *b*, hedef tabanı *h* olan bir sayı önce kaynak tabandan **kesin** bir değere çevrilir (tam kısım bir tam sayı, kesirli kısım `pay / b^m` kesri olarak tutulur, böylece ondalıklı sayı yuvarlaması oluşmaz), sonra bu değer hedef tabana çevrilir. Tam ve kesirli kısım ayrı ayrı işlenir.

1.1 Tam kısım: ardışık bölme

Hedef tabanda tam kısım

$$T = d_k \cdot h^k + \ldots + d_2 \cdot h^2 + d_1 \cdot h^1 + d_0 \cdot h^0$$

biçimindedir. Bu ifadeyi *h* ile bölersek:

$$T = h \cdot (d_k \cdot h^{k-1} + \ldots + d_1) + \mathbf{d_0}$$

Yani **ilk kalan $\mathbf{d_0}$**, yani en düşük ağırlıklı (h^0) basamaktır. Bölüm, aynı işlemle bir sonraki kalanı (d_1, ağırlık h^1) verir. Kalanlar düşük ağırlıktan yüksek ağırlığa doğru çıkar. Sayıyı yazarken ise en yüksek ağırlıklı basamak solda olmalıdır. **Bu yüzden kalanlar ters sırada yazılır.**

Örnek: 45 → onaltılı

Adım	İşlem	Bölüm	Kalan	Basamak ağırlığı
1	45 ÷ 16	2	13 = D	16^0
2	2 ÷ 16	0	2	16^1

Ters sırada okunursa **2D**. Kontrol: $2 \cdot 16^1 + 13 \cdot 16^0 = 32 + 13 = 45$.

1.2 Kesirli kısım: ardışık çarpma

Hedef tabanda kesirli kısım

$$F = d_{-1} \cdot h^{-1} + d_{-2} \cdot h^{-2} + d_{-3} \cdot h^{-3} + \ldots$$

biçimindedir. Bu ifadeyi *h* ile çarparsak:

$$F \cdot h = \mathbf{d_{-1}} + (d_{-2} \cdot h^{-1} + d_{-3} \cdot h^{-2} + \ldots)$$

Parantez içindeki kısım 1'den küçüktür, bu yüzden çarpımın **tam kısmı $\mathbf{d_{-1}}$**, yani noktadan hemen sonraki en yüksek ağırlıklı (h^{-1}) basamaktır. Kalan kesir yeniden çarpılınca d_{-2} (h^{-2})

çıkar. Basamaklar yüksek ağırlıktan düşük ağırlığa doğru, noktadan sağa doğru yazılma sırasıyla çıkar. **Bu yüzden işlem sırasıyla yazılır.**

Örnek: 0.40625 → onaltılı

Adım	İşlem	Sonuç	Basamak	Ağırlık
1	0.40625 × 16	6.5	6	16^{-1}
2	0.5 × 16	8.0	8	16^{-2}
–	kalan kesir 0		dur, **kesin**	

Sonuç **0.68**. Kontrol: 6/16 + 8/256 = 0.375 + 0.03125 = 0.40625.

1.3 Örnek girişin tam çözümü: $(101101.01101)_2$ → onaltılı

- Tam kısım: $101101_2 = 1{\cdot}32 + 0{\cdot}16 + 1{\cdot}8 + 1{\cdot}4 + 0{\cdot}2 + 1{\cdot}1 = 45$ → **2D** (1.1'deki tablo)
- Kesir: $.01101_2 = 0{\cdot}½ + 1{\cdot}¼ + 1{\cdot}⅛ + 0{\cdot}1/16 + 1{\cdot}1/32 = 13/32 = 0.40625$ → **.68** (1.2'deki tablo)
- **Sonuç: $(101101.01101)_2 = (2D.68)_{16}$, kesin.**

2. Giriş kontrolü

Dönüşüme başlamadan önce `giris_kontrolu()` şu durumlarda girdiyi reddeder:

Kural	Örnek	Sonuç
Taban 2, 8, 10, 16 dışında	taban 3	RED
Boş veya yalnızca boşluk	`""`	RED
Yalnızca nokta, rakam yok	`"."`	RED
Birden çok kesir ayracı	`1.0.1`	RED
Rakam olmayan karakter (işaret dahil)	`-5`, `1G`	RED
Rakamın değeri ≥ taban	`102` (taban 2), `128` (taban 8)	RED
Noktanın bir yanında rakam yok	`.5`, `5.`	RED
64 bitlik değişkene sığmayan sayı	`FFFFFFFFFFFFFFFF` (17 hane)	RED

Rakam değeri, `0123456789ABCDEF` tablosundaki sırasıdır. Bir rakam ancak değeri tabandan küçükse geçerlidir: 2 tabanında yalnızca 0–1, 8 tabanında 0–7 kullanılabilir. Hata mesajı hatalı karakteri ve konumunu belirtir.

Talimatta belirtilmeyen konularda verdiğim kararlar:

- **Noktanın iki yanında da rakam olmalı.** Ders notlarındaki bütün örnekler $(0.65)_{10}$, $(0.975)_{10}$ gibi noktadan önce 0 ile yazılmıştır. Bu yüzden `.5` ve `5.` belirsiz yazım sayılıp reddedildi; doğru yazımları `0.5` ve `5`'tir.
- **Onaltılıda küçük harf kabul edilir.** Ders notları büyük harf kullanır (AE, 1A7), ama `ff` ile `FF` aynı değeri gösterir ve bunu reddetmek için bir gerekçe yoktur. Çıktı her zaman büyük harfle yazılır.

- Ayrıca C’de değişkenler sınırlı olduğundan, 64 bite sığmayan girdiler yanlış sonuç vermek yerine reddedilir.

3. Kesinlik bilgisi

Kesirli dönüşüm en çok **8 hedef basamak** üretir.

- Kalan kesir 8 basamak içinde 0 olursa sonuç **kesin**dir ve = ile gösterilir.
- 8 basamakta hâlâ sıfırdan farklı kalan varsa işlem kesilir, sonuç **yaklaşık**tır ve ≈ ile gösterilir. Kesmenin hatası her zaman son basamağın ağırlığından küçüktür: hata < h^{-8}.

Örnek: 0.1_{10} → ikili. Çarpmalar sırasıyla 0, 0, 0, 1, 1, 0, 0, 1 verir ve kalan 0.6 olduğundan dönüşüm bitmez (0011 grubu sonsuza kadar tekrar eder). $(0.1)_{10} \approx (0.00011001)_2 = 1/16 + 1/32 + 1/256 = 0.09765625$. Hata $0.00234375 < 2^{-8} = 0.00390625$.

4. Kod

Kod C dilinde yazılmıştır (tam metni Ek’te). Ana dönüşüm hazır fonksiyonlara yaptırılmamıştır: `strtol`, `strtoull`, `printf("%x")` / `"%o"` gibi taban dönüştüren fonksiyonlar kullanılmaz. Yalnızca `unsigned long long` tam sayılarla `+ × / %` işlemleri ve rakam tablosu kullanılır. Kesirli kısım `double` yerine pay/payda olarak tutulduğundan yuvarlama hatası oluşmaz.

Fonksiyon	Görevi
`giris_kontrolu`	Bölüm 2’deki kuralları uygular
`metinden_degere`	Kaynak metni Horner yöntemiyle tam sayıya, kesri pay/payda çiftine çevirir
`tam_kismi_cevir`	Ardışık bölme, kalanları ters çevirir
`kesri_cevir`	Ardışık çarpma, en çok 8 basamak, kesinlik bilgisini döndürür
`donustur`	Hepsini birleştirir, sonucu ve kesinlik bilgisini döndürür
`testleri_calistir`	Test tablosunu çalıştırır, beklenen ve gerçekleşen sonuçları karşılaştırır

Programın çıktısında yaklaşık sonuçlar, düz metinde ≈ yazılamadığı için ~ ile gösterilir.

5. Test tablosu

Beklenen sonuçlar kod çalıştırılmadan önce elle hesaplandı. “Gerçekleşen” sütunu programın çıktısıdır.

No	Girdi	Kaynak → Hedef	Ne sınanıyor	Beklenen	Gerçekleşen	Durum
T1	101101.01101	2 → 16	örnek giriş	= 2D.68	= 2D.68	Geçti
T2	0	10 → 2	sıfır	= 0	= 0	Geçti
T3	0.625	10 → 2	yalnızca kesir, sonlu	= 0.101	= 0.101	Geçti
T4	0.1	10 → 2	yalnızca kesir, sonsuz	≈ 0.00011001	≈ 0.00011001	Geçti
T5	000173	10 → 16	başında sıfır	= AD	= AD	Geçti
T6	AD.A	16 → 10	onaltılı → onluk	= 173.625	= 173.625	Geçti
T7	755	8 → 2	sekizli → ikili	= 111101101	= 111101101	Geçti
T8	0.2	10 → 8	sekizli, yaklaşık	≈ 0.14631463	≈ 0.14631463	Geçti
T9	ff.8	16 → 2	küçük harf	= 11111111.1	= 11111111.1	Geçti
T10	102	2 → 10	geçersiz rakam	RED	RED	Geçti
T11	128	8 → 10	geçersiz rakam	RED	RED	Geçti
T12	(boş)	10 → 2	boş girdi	RED	RED	Geçti
T13	1.0.1	2 → 10	iki ayraç	RED	RED	Geçti
T14	-5	10 → 2	negatif işaret	RED	RED	Geçti
T15	12	3 → 10	desteklenmeyen taban	RED	RED	Geçti
T16	.5	10 → 2	noktadan önce rakam yok	RED	RED	Geçti
T17	5.	10 → 2	noktadan sonra rakam yok	RED	RED	Geçti
T18	FFFFFFFFFFFFFFFF	16 → 10	64 bite sığmayan sayı	RED	RED	Geçti

Sonuç: 18/18 test geçti.

Elle hesap notları:

- T3: 0.625×2 = 1.25 → 1; 0.25×2 = 0.5 → 0; 0.5×2 = 1.0 → 1, kalan 0, kesin → 0.101
- T5: baştaki sıfırların ağırlığı 0'dır, değer 173. 173 ÷ 16 = 10 kalan 13 (D); 10 ÷ 16 = 0 kalan 10 (A) → AD
- T7: her sekizli rakam 3 bit: 7 = 111, 5 = 101, 5 = 101 → 111101101
- T8: 0.2×8 = 1.6 → 1; 0.6×8 = 4.8 → 4; 0.8×8 = 6.4 → 6; 0.4×8 = 3.2 → 3; ardından 1, 4, 6, 3 tekrar eder → ≈ 0.14631463

6. Bağımsız kontrol hesabı (basamak ağırlıklarıyla)

Programın kullandığı yöntemden (bölme/çarpma) **farklı** iki yöntemle T1 ve T6 denetlendi.

T1, yöntem A: iki tarafı da onluya çevirmek

- Sol: $101101.01101_2 = 2^5 + 2^3 + 2^2 + 2^0 + 2^{-2} + 2^{-3} + 2^{-5} = 32 + 8 + 4 + 1 + 0.25 + 0.125 + 0.03125 =$ **45.40625**
- Sağ: $2D.68_{16} = 2\cdot16^1 + 13\cdot16^0 + 6\cdot16^{-1} + 8\cdot16^{-2} = 32 + 13 + 0.375 + 0.03125 =$ **45.40625** ✓

T1, yöntem B: dörtlü gruplama ($16 = 2^4$ olduğu için her onaltılı basamak tam olarak 4 bit)

- Tam kısım noktadan sola: `00`101101 → 0010 | 1101 → 2 | D (soldaki iki 0 eklendi)
- Kesir noktadan sağa: 01101`000` → 0110 | 1000 → 6 | 8 (sağdaki üç 0 eklendi)
- Sonuç 2D.68 ✓. Eklenen sıfırlar değeri değiştirmez, çünkü ağırlıkları ne olursa olsun katsayıları 0'dır.

T6: $AD.A_{16} = 10\cdot16 + 13 + 10/16 = 160 + 13 + 0.625 =$ **173.625** ✓

7. Düşünme sorusu

Başka tabana çevirip geri dönünce aynı değeri bulmak doğruluğu kanıtlar mı? Hayır.

Gidiş-dönüş testi yalnızca iki fonksiyonun **birbirinin tersi** olduğunu gösterir, ikisinin de **doğru** olduğunu göstermez. Program iki yönde de aynı hatayı yaparsa hatalar birbirini götürür.

Karşı örnek (koddaki `bozuk_tam_cevir` ve `bozuk_geri_cevir` fonksiyonları):

- Bozuk dönüştürücü kalanları **ters çevirmeyi unutuyor**: 6_{10} → 6÷2 = 3 k0, 3÷2 = 1 k1, 1÷2 = 0 k1 → "011"
- Bozuk geri dönüştürücü rakamları **sağdan sola okuyor**: "011" → 1, 1, 0 → 1·4 + 1·2 + 0 = 6
- Gidiş-dönüş 6 → "011" → 6 verdiği için test **geçer**, ama 011_2 gerçekte 0·4 + 1·2 + 1·1 = **3**'tür. Doğru cevap 110_2'dir.

Program çıktısı: `6 -> '011' -> 6`, `11 -> '1101' -> 11`, `13 -> '1011' -> 13`. Hepsi gidiş-dönüşü geçer ve hepsi yanlıştır.

Ayrıca tersi de geçerlidir: **doğru** bir program gidiş-dönüşte başarısız olabilir. 0.1_{10} → 0.00011001_2 (kesilmiş) → geri çevrilince 0.09765625 olur, 0.1 değil. Bu hata değil, kesmenin doğal sonucudur.

Sonuç: Gidiş-dönüş testi doğruluğu kanıtlamaz. Doğruluk, sonucun **bağımsız bir yöntemle** (basamak ağırlıklarıyla onluya çevirme veya bit gruplama) denetlenmesiyle gösterilir. Bölüm 6'da bu yapılmıştır.

8. Kısa cevap ve doğrulama notları

- **İlk tahminim:** $(101101.01101)_2 = (2D.68)_{16}$ ve dönüşüm kesin olmalı, çünkü 2 ve 16 tabanları arasındaki her dönüşüm sonludur ($16 = 2^4$).
- **Bağımsız kontrolüm:** İki taraf da onluya çevrildi: 45.40625 = 45.40625. Dörtlü gruplama ile de 0010 1101 . 0110 1000 = 2D.68 bulundu.
- **Karşı örneğim:** Kalanları ters çevirmeyen ve rakamları ters okuyan bir program 6 → "011" → 6 gidiş-dönüşünü geçer, ama $011_2 = 3 \neq 6$.

- **YZ önerisini değerlendirmem / beyanım:** *(Bu alanı kendin, kendi cümlelerinle doldur. Ne kullandığını ve hangi sonucu nasıl elle kontrol ettiğini yaz. Örneğin: "Kod taslağı için YZ desteği aldım. T1, T3, T5 ve T6 sonuçlarını kod çalıştırmadan önce elle hesapladım ve ağırlıklarla doğruladım.")*

YZ desteği ve doğrulama

Bu yaprakta algoritma açıklaması, C kodu ve test tablosu için Claude'dan yardım aldım. İlk öneri kodun Python ile yazılmasıydı. Dersin dili belirtilmediği için kodu C'ye çevirttim ve taban dönüştüren hazır bir fonksiyon (`strtol`, `printf("%x")` gibi) kullanılmadığını kontrol ettim.

İlk sürüm `.5` ve `5.` girdilerini kabul ediyordu. Ders notlarındaki bütün örneklerde noktadan önce 0 yazıldığı için bu kuralı değiştirdim, artık bu girdiler reddediliyor.

Doğrulama: Testlerin beklenen sonuçlarını koddan bağımsız olarak hesapladım ve programın 18 testin hepsini geçtiğini gördüm. Örnek girişi ayrıca basamak ağırlıklarıyla ve dörtlü gruplamayla kontrol ettim (Bölüm 6).

Kod: deney1_donusturucu.c

```c
#include <stdio.h>
#include <string.h>
#include <limits.h>

#define MAKS_KESIR_BASAMAGI 8
#define MAKS_GIRDI 64

static const char RAKAMLAR[] = "0123456789ABCDEF";

typedef struct {
    char tam[MAKS_GIRDI + 1];
    char kesir[MAKS_KESIR_BASAMAGI + 1];
    int kesin;
} Sonuc;

int rakam_degeri(char ch)
{
    if (ch >= 'a' && ch <= 'f')
        ch = ch - 'a' + 'A';
    for (int d = 0; d < 16; d++)
        if (RAKAMLAR[d] == ch)
            return d;
    return -1;
}

int gecerli_taban(int taban)
{
    return taban == 2 || taban == 8 || taban == 10 || taban == 16;
}

/* Hata yoksa 0, varsa hata mesajını hata[] içine yazıp 1 döndürür. */
int giris_kontrolu(const char *metin, int taban, char *hata)
{
    int n = (int)strlen(metin);
    int nokta_sayisi = 0, nokta_yeri = -1;

    if (!gecerli_taban(taban)) {
        sprintf(hata, "gecersiz taban %d (2, 8, 10 veya 16 olmali)", taban);
        return 1;
    }
    if (n == 0) {
        sprintf(hata, "bos girdi");
        return 1;
```

```
            }
            if (n > MAKS_GIRDI) {
                sprintf(hata, "girdi cok uzun");
                return 1;
            }
            for (int i = 0; i < n; i++) {
                if (metin[i] == '.') {
                    nokta_sayisi++;
                    nokta_yeri = i;
                    continue;
                }
                int d = rakam_degeri(metin[i]);
                if (d < 0) {
                    sprintf(hata, "'%c' rakam degil (konum %d)", metin[i], i);
                    return 1;
                }
                if (d >= taban) {
                    sprintf(hata, "'%c' rakami %d tabaninda gecersiz (konum %d)", metin[i], taban, i);
                    return 1;
                }
            }
            if (nokta_sayisi > 1) {
                sprintf(hata, "birden cok kesir ayraci");
                return 1;
            }
            /* Ders notlarindaki yazim gibi noktanin iki yaninda da rakam olmali: ".5" ve "5."
reddedilir. */
            if (nokta_sayisi == 1 && (nokta_yeri == 0 || nokta_yeri == n - 1)) {
                sprintf(hata, "noktanin iki yaninda da rakam olmali");
                return 1;
            }
            return 0;
        }

        /*
         * Kaynak metni kesin degere cevirir. Kesir, yuvarlama hatasi olmamasi icin
         * pay / payda (payda = taban^m) olarak tutulur.
         */
        int metinden_degere(const char *metin, int taban, unsigned long long *T,
                            unsigned long long *pay, unsigned long long *payda, char *hata)
        {
            int i = 0;
            *T = 0;
            *pay = 0;
            *payda = 1;

            for (; metin[i] != '\0' && metin[i] != '.'; i++) {
                int d = rakam_degeri(metin[i]);
                if (*T > (ULLONG_MAX - d) / taban) {
                    sprintf(hata, "tam kisim cok buyuk");
                    return 1;
                }
                *T = *T * taban + d;            /* Horner: T = (...(d_k*b + d_k-1)*b + ...)*b + d_0 */
            }
            if (metin[i] == '.')
                i++;
            for (; metin[i] != '\0'; i++) {
                /* kesri_cevir payi 16 ile carpacagi icin payda 16 kat buyumeye yer birakmali */
                if (*payda > ULLONG_MAX / taban / 16) {
                    sprintf(hata, "kesirli kisim cok uzun");
                    return 1;
                }
                *pay = *pay * taban + rakam_degeri(metin[i]);
                *payda = *payda * taban;
            }
            return 0;
        }

        void tam_kismi_cevir(unsigned long long T, int hedef, char *cikti)
        {
            char kalanlar[MAKS_GIRDI * 4 + 1];
            int k = 0;
```

```c
    if (T == 0) {
        strcpy(cikti, "0");
        return;
    }
    while (T > 0) {
        kalanlar[k++] = RAKAMLAR[T % hedef];   /* ilk kalan en dusuk agirlikli basamak (h^0) */
        T /= hedef;
    }
    for (int i = 0; i < k; i++)                /* bu yuzden ters sirada yaziliyor */
        cikti[i] = kalanlar[k - 1 - i];
    cikti[k] = '\0';
}

/* Kesin ise 1, 8 basamakta bitmediyse 0 dondurur. */
int kesri_cevir(unsigned long long pay, unsigned long long payda, int hedef, char *cikti)
{
    int k = 0;
    while (pay != 0 && k < MAKS_KESIR_BASAMAGI) {
        pay *= hedef;
        cikti[k++] = RAKAMLAR[pay / payda];    /* tam kisim, h^-1'den baslayarak sirayla basamak
*/
        pay %= payda;
    }
    cikti[k] = '\0';
    return pay == 0;
}

int donustur(const char *metin, int kaynak, int hedef, Sonuc *s, char *hata)
{
    unsigned long long T, pay, payda;

    if (!gecerli_taban(hedef)) {
        sprintf(hata, "gecersiz hedef taban %d", hedef);
        return 1;
    }
    if (giris_kontrolu(metin, kaynak, hata))
        return 1;
    if (metinden_degere(metin, kaynak, &T, &pay, &payda, hata))
        return 1;
    tam_kismi_cevir(T, hedef, s->tam);
    s->kesin = kesri_cevir(pay, payda, hedef, s->kesir);
    return 0;
}

void sonucu_yaz(const Sonuc *s, char *cikti)
{
    if (s->kesir[0] != '\0')
        sprintf(cikti, "%c %s.%s", s->kesin ? '=' : '~', s->tam, s->kesir);
    else
        sprintf(cikti, "%c %s", s->kesin ? '=' : '~', s->tam);
}

/* Beklenen sonuclar program yazilmadan once elle hesaplandi. "RED" = reddedilmeli. */
typedef struct {
    const char *girdi;
    int kaynak, hedef;
    const char *beklenen;
    const char *aciklama;
} Test;

static const Test TESTLER[] = {
    {"101101.01101", 2, 16, "= 2D.68",       "ornek giris"},
    {"0",            10, 2, "= 0",           "sifir"},
    {"0.625",        10, 2, "= 0.101",       "yalnizca kesir, sonlu"},
    {"0.1",          10, 2, "~ 0.00011001",  "yalnizca kesir, sonsuz"},
    {"000173",       10, 16, "= AD",         "basinda sifir"},
    {"AD.A",         16, 10, "= 173.625",    "onaltilidan onluya"},
    {"755",          8, 2,  "= 111101101",   "sekizliden ikiliye"},
    {"0.2",          10, 8, "~ 0.14631463",  "sekizli, yaklasik"},
    {"ff.8",         16, 2, "= 11111111.1",  "kucuk harf"},
    {"102",          2, 10, "RED",           "gecersiz rakam"},
```

```
            {"128",          8, 10, "RED",           "gecersiz rakam"},
            {"",             10, 2, "RED",           "bos girdi"},
            {"1.0.1",        2, 10, "RED",           "iki ayrac"},
            {"-5",           10, 2, "RED",           "negatif isaret"},
            {"12",           3, 10, "RED",           "desteklenmeyen taban"},
            {".5",           10, 2, "RED",           "noktadan once rakam yok"},
            {"5.",           10, 2, "RED",           "noktadan sonra rakam yok"},
            {"FFFFFFFFFFFFFFFF", 16, 10, "RED",     "64 bite sigmayan sayi"},
        };

        int testleri_calistir(void)
        {
            int n = sizeof(TESTLER) / sizeof(TESTLER[0]), gecen = 0;
            char gercek[200], hata[200], tirnakli[MAKS_GIRDI + 3];
            Sonuc s;

            printf("%-4s %-20s %4s %4s  %-16s %-16s %-6s %s\n",
                   "No", "Girdi", "Kay", "Hdf", "Beklenen", "Gerceklesen", "Durum", "Aciklama");
            for (int i = 0; i < n; i++) {
                const Test *t = &TESTLER[i];
                if (donustur(t->girdi, t->kaynak, t->hedef, &s, hata))
                    strcpy(gercek, "RED");
                else
                    sonucu_yaz(&s, gercek);
                int ok = strcmp(gercek, t->beklenen) == 0;
                gecen += ok;
                sprintf(tirnakli, "\"%s\"", t->girdi);
                printf("T%-3d %-20s %4d %4d  %-16s %-16s %-6s %s\n", i + 1, tirnakli,
                       t->kaynak, t->hedef, t->beklenen, gercek, ok ? "GECTI" : "KALDI", t->aciklama);
            }
            printf("\n%d/%d test gecti.\n", gecen, n);
            return gecen == n;
        }

        /* Dusunme sorusu: iki yonde AYNI hatayi yapan bozuk program. */
        void bozuk_tam_cevir(unsigned long long T, int hedef, char *cikti)
        {
            int k = 0;
            if (T == 0)
                cikti[k++] = '0';
            while (T > 0) {
                cikti[k++] = RAKAMLAR[T % hedef];      /* HATA: kalanlar ters cevrilmiyor */
                T /= hedef;
            }
            cikti[k] = '\0';
        }

        unsigned long long bozuk_geri_cevir(const char *metin, int taban)
        {
            unsigned long long T = 0;
            for (int i = (int)strlen(metin) - 1; i >= 0; i--)
                T = T * taban + rakam_degeri(metin[i]);   /* AYNI HATA: rakamlar sagdan sola okunuyor */
            return T;
        }

        int main(void)
        {
            char hata[200], cikti[200];
            Sonuc s;

            printf("== Ornek giris ==\n");
            if (donustur("101101.01101", 2, 16, &s, hata) == 0) {
                sonucu_yaz(&s, cikti);
                printf("(101101.01101)2 -> 16 tabani: %s  [%s]\n\n", cikti, s.kesin ? "kesin" :
"yaklasik");
            }

            printf("== Hata mesaji ornekleri ==\n");
            const char *hatali[] = {"102", "1.0.1", ""};
            int tabanlar[] = {2, 2, 10};
            for (int i = 0; i < 3; i++)
                if (donustur(hatali[i], tabanlar[i], 10, &s, hata))
```

```
                    printf("\"%s\" (taban %d): REDDEDILDI, %s\n", hatali[i], tabanlar[i], hata);

            printf("\n== Test tablosu ==\n");
            testleri_calistir();

            printf("\n== Dusunme sorusu: gidis-donus bozuk programi yakalayamaz ==\n");
            unsigned long long sayilar[] = {6, 11, 13};
            for (int i = 0; i < 3; i++) {
                char bozuk[80], dogru[80];
                unsigned long long gercek_deger, p, q;
                bozuk_tam_cevir(sayilar[i], 2, bozuk);
                tam_kismi_cevir(sayilar[i], 2, dogru);
                metinden_degere(bozuk, 2, &gercek_deger, &p, &q, hata);
                printf("%llu -> bozuk ikili '%s' -> geri %llu  (dogru ikili %s, '%s' gercekte =
%llu)\n",
                       sayilar[i], bozuk, bozuk_geri_cevir(bozuk, 2), dogru, bozuk, gercek_deger);
            }
            return 0;
        }
```

Program çıktısı

```
== Ornek giris ==
(101101.01101)2 -> 16 tabani: = 2D.68  [kesin]

== Hata mesaji ornekleri ==
"102" (taban 2): REDDEDILDI, '2' rakami 2 tabaninda gecersiz (konum 2)
"1.0.1" (taban 2): REDDEDILDI, birden cok kesir ayraci
"" (taban 10): REDDEDILDI, bos girdi

== Test tablosu ==
No   Girdi                 Kay  Hdf  Beklenen         Gerceklesen      Durum  Aciklama
T1   "101101.01101"          2   16  = 2D.68          = 2D.68          GECTI  ornek giris
T2   "0"                     10    2  = 0              = 0              GECTI  sifir
T3   "0.625"                 10    2  = 0.101          = 0.101          GECTI  yalnizca kesir, sonlu
T4   "0.1"                   10    2  ~ 0.00011001     ~ 0.00011001     GECTI  yalnizca kesir, sonsuz
T5   "000173"                10   16  = AD             = AD             GECTI  basinda sifir
T6   "AD.A"                  16   10  = 173.625        = 173.625        GECTI  onaltilidan onluya
T7   "755"                    8    2  = 111101101      = 111101101      GECTI  sekizliden ikiliye
T8   "0.2"                   10    8  ~ 0.14631463     ~ 0.14631463     GECTI  sekizli, yaklasik
T9   "ff.8"                  16    2  = 11111111.1     = 11111111.1     GECTI  kucuk harf
T10  "102"                    2   10  RED              RED              GECTI  gecersiz rakam
T11  "128"                    8   10  RED              RED              GECTI  gecersiz rakam
T12  ""                      10    2  RED              RED              GECTI  bos girdi
T13  "1.0.1"                  2   10  RED              RED              GECTI  iki ayrac
T14  "-5"                    10    2  RED              RED              GECTI  negatif isaret
T15  "12"                     3   10  RED              RED              GECTI  desteklenmeyen taban
T16  ".5"                    10    2  RED              RED              GECTI  noktadan once rakam yok
T17  "5."                    10    2  RED              RED              GECTI  noktadan sonra rakam yok
T18  "FFFFFFFFFFFFFFFF"      16   10  RED              RED              GECTI  64 bite sigmayan sayi

18/18 test gecti.

== Dusunme sorusu: gidis-donus bozuk programi yakalayamaz ==
6 -> bozuk ikili '011' -> geri 6  (dogru ikili 110, '011' gercekte = 3)
11 -> bozuk ikili '1101' -> geri 11  (dogru ikili 1011, '1101' gercekte = 13)
13 -> bozuk ikili '1011' -> geri 13  (dogru ikili 1101, '1011' gercekte = 11)
```

Deney Yaprağı 2 – Ek: Sınırlı Bit Sayısıyla Ölçüm Gösterimi Tasarımı

Problem: Cihaz 0 ile 1000 arasındaki değerleri (iki uç dahil) 0.125 adımlarla gösterecek. İkili noktanın yeri sabit, işaret biti yok, nokta bit sayısına katılmıyor.

1. Kesirli kısım için en az bit sayısı

Kesirli bitlerin ağırlıkları $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, … şeklindedir.

- $0.125 = 2^{-3}$ olduğu için gösterimde ağırlığı 2^{-3} olan bir bit **bulunmak zorundadır**.
- 2 kesirli bitle en küçük adım $2^{-2} = 0.25$ olur. Bu durumda 0.125 ve 0.375 gösterilemez, yani 2 bit yetmez.
- 3 kesirli bitle .000 … .111 arası 8 durum vardır: 0, 0.125, 0.25, …, 0.875. Bunlar 0.125'in bütün katlarıdır.

Kesirli kısım: en az 3 bit.

2. Tam kısım ve toplam bit sayısı

En büyük tam değer 1000'dir. n bitlik bir tam kısım 0 … $2^n - 1$ aralığını gösterir.

- $2^9 - 1 = 511 < 1000$ olduğu için 9 bit yetmez.
- $2^{10} - 1 = 1023 \geq 1000$ olduğu için 10 bit yeter.

Tam kısım: en az 10 bit. Toplam: 10 + 3 = 13 bit.

3. Toplam 12 bit yeterli midir? (iki yöntem)

Yöntem 1, temsil edilebilen en büyük değer: 12 bitte 0.125 adımı korunacaksa 3 bit kesire gider ve tam kısma 9 bit kalır. En büyük değer $111111111.111_2 = 511.875 < 1000$ olur. Tam kısma 10 bit verilirse kesire 2 bit kalır ve adım 0.25'e çıkar, 0.125 adımı bozulur. **İki durumda da 12 bit yetmez.**

Yöntem 2, farklı gösterimlerin sayısı: Gösterilmesi gereken değer sayısı 1000 / 0.125 + 1 = 8000 + 1 = **8001**'dir (0 dahil). 12 bit en çok $2^{12} =$ **4096** farklı durum verir. 4096 < 8001 olduğu için **12 bit yetmez**. 13 bit $2^{13} = 8192 \geq 8001$ durum verir ve yeter.

İki yöntem de aynı sonucu verir: **12 bit yetersizdir, gereken en az bit sayısı 13'tür.**

4. Tasarlanan en küçük gösterim

13 bit: 10 bit tam kısım + 3 bit kesirli kısım (sabit noktalı "10.3" biçimi)

$b_9\ b_8\ b_7\ b_6\ b_5\ b_4\ b_3\ b_2\ b_1\ b_0\ .\ b_{-1}\ b_{-2}\ b_{-3}$

- Ağırlıklar: 512, 256, …, 2, 1 . 0.5, 0.25, 0.125
- Aralık: 0 … 1023.875 (gereken 0 … 1000 aralığını kapsar)
- Adım (en küçük bitin ağırlığı): $2^{-3} = 0.125$

5. İstenen değerler (13 bit, baştaki ve sondaki sıfırlar korunmuştur)

Değer	Tam kısım (10 bit)	Kesir (3 bit)	13 bitlik gösterim
0	0000000000	000	0000000000.000
0.125	0000000000	001	0000000000.001
173.625	0010101101	101	0010101101.101
1000	1111101000	000	1111101000.000

Hesaplar:

- 173 = 128 + 32 + 8 + 4 + 1 → 10101101, 10 bite tamamlanınca 0010101101
- 0.625 = 0.5 + 0.125 → .101
- 1000 = 512 + 256 + 128 + 64 + 32 + 8 → 1111101000
- Sınır kontrolü: 1000 en büyük değerdir ve 10 bite tam sığar. 0 en küçük değerdir ve tamamı sıfırdır.

6. 173.625 sekizli ve onaltılı

Sekizli (noktadan itibaren 3'lü gruplar): 010 101 101 . 101 → **$(255.5)_8$** Kontrol: 2·64 + 5·8 + 5 + 5/8 = 128 + 40 + 5 + 0.625 = **173.625** ✓

Onaltılı (noktadan itibaren 4'lü gruplar): 1010 1101 . 1010 → **$(AD.A)_{16}$** Kontrol: 10·16 + 13 + 10/16 = 160 + 13 + 0.625 = **173.625** ✓

İkili kontrol: 128 + 32 + 8 + 4 + 1 + 0.5 + 0.125 = **173.625** ✓

Üç gösterim de aynı değeri verir.

Düşünme sorusu

Adım 0.1 olsaydı bütün değerleri sonlu ikili kesirle tam gösterebilir miydik? Hayır.

Sonlu bir ikili kesir, ağırlıkları 2^{-1}, 2^{-2}, …, 2^{-k} olan bitlerin toplamıdır. Bu toplam her zaman $m / 2^k$ biçimindedir, yani paydası 2'nin bir kuvvetidir. 0.1 = 1/10 olduğundan paydasında 5 çarpanı vardır ve hiçbir 2^k, 10'a bölünmez. Bu yüzden 0.1 sonlu ikili kesirle yazılamaz:

$0.1_{10} = 0.000110011001100\ldots_2$ (0011 sonsuza kadar tekrar eder)

0.1, 0.2, 0.3 gibi değerler ancak **yaklaşık** gösterilebilir. 0.5 gibi paydası 2'nin kuvveti olan değerler ise yine tam gösterilir. 0.125 adımında sorun yoktur, çünkü 0.125 = 1/8 = $1/2^3$.

YZ desteği ve doğrulama

Bit sayısı hesabında Claude'dan yardım aldım. Önerilen 13 bitlik sonucu iki ayrı yöntemle kontrol ettim: gösterilecek değer sayısı ($8001 > 2^{12} = 4096$) ve 12 bitte gösterilebilen en büyük değer ($511.875 < 1000$).

173.625'in ikili, sekizli ve onaltılı gösterimlerini ayrı ayrı onluya çevirdim ve üçünün de aynı değeri verdiğini doğruladım.

Deney Yaprağı 3 – Ek: Sıfır Ekleme ve Basamak Değerinin Korunması

İncelenecek sayı: $(101101.01101)_2$, sekizli ve onaltılı sistemlere dönüştürülecek.

1. Gruplar ikili noktadan başlayarak (eklenen sıfırlar [köşeli parantez] içinde)

Kural: Tam kısım noktadan **sola** doğru, kesirli kısım noktadan **sağa** doğru gruplanır. Eksik kalan grup tam kısımda **solda**, kesirli kısımda **sağda** sıfırla tamamlanır.

Sekizli (3'lü gruplar, $8 = 2^3$):

	Tam kısım		**Kesirli kısım**	
Gruplar	101	101	011	01**[0]**
Sekizli rakam	5	5	3	2

→ $\mathbf{(55.32)_8}$. Tam kısım 6 bit olduğundan tam bölündü. Kesirli kısma sağdan bir sıfır eklendi.

Onaltılı (4'lü gruplar, $16 = 2^4$):

	Tam kısım		**Kesirli kısım**	
Gruplar	**[00]**10	1101	0110	1**[000]**
Onaltılı rakam	2	D	6	8

→ $\mathbf{(2D.68)_{16}}$. Tam kısmın soluna iki, kesirli kısmın sağına üç sıfır eklendi.

2. Eklenen sıfırlar neden değeri değiştirmez?

Bir sayının değeri, her rakamın kendi basamak ağırlığıyla çarpımlarının toplamıdır:

değer = $\Sigma\, d_i \cdot 2^i$

- **Tam kısmın soluna** eklenen sıfırlar, mevcut en yüksek basamaktan daha büyük ağırlıklara (2^6, 2^7 …) yerleşir. Katsayıları 0 olduğu için toplama $0\cdot2^6 + 0\cdot2^7 = 0$ eklerler. **Mevcut bitlerin ağırlığı değişmez.**
- **Kesirli kısmın sağına** eklenen sıfırlar, daha küçük ağırlıklara (2^{-6}, 2^{-7} …) yerleşir. Bunlar da toplama 0 ekler. **Mevcut bitlerin noktaya uzaklığı, dolayısıyla ağırlığı aynı kalır.**

Örnek: tam kısım 00101101 = $0\cdot128 + 0\cdot64 + 32 + 8 + 4 + 1 = 45$. Kesir .01101000 = ¼ + ⅛ + 1/32 + 0 + 0 + 0 = 0.40625.

Kısaca: sıfırlar mevcut bitlerin **noktaya göre konumunu** değiştirmediği sürece değer değişmez.

3. Hatalı gruplama: 01101 yerine 00001101

Öğrenci kesirli kısmın **soluna** (noktanın hemen yanına) sıfır eklemiştir:

.**[000]**01101 → 0000 | 1101 → **.0D**

- Bulunan onaltılı: **$(2D.0D)_{16}$**
- Onluk değer: 2·16 + 13 + 0/16 + 13/256 = 45 + 0.05078125 = **45.05078125**
- Doğru değer: 45.40625. Fark: **0.35546875**

Sebep: Soldaki sıfırlar mevcut bitleri noktadan 3 basamak uzaklaştırır. Her bitin ağırlığı $2^3 = 8$ kat küçülür: kesir 0.40625 / 8 = 0.05078125 olur.

4. "Eksik gruplar her zaman sola sıfır eklenerek tamamlanır" iddiası yanlıştır

Karşı örnek: $(0.1)_2 = 1/2 = 0.5$

- Sola sıfır eklenirse: .0001 → $(0.1)_{16} = 1/16 =$ **0.0625** ×
- Sağa sıfır eklenirse: .1000 → $(0.8)_{16} = 8/16 =$ **0.5** ✓

Doğru kural: **tam kısım solda, kesirli kısım sağda** tamamlanır. Yani her zaman noktadan **uzak** tarafa sıfır eklenir.

5. Eşitlik doğrulaması (her gösterim ayrı ayrı onluya çevrildi)

Gösterim	Basamak ağırlıklarıyla açılım	Onluk
$(101101.01101)_2$	32 + 8 + 4 + 1 + 0.25 + 0.125 + 0.03125	**45.40625**
$(55.32)_8$	5·8 + 5 + 3/8 + 2/64 = 40 + 5 + 0.375 + 0.03125	**45.40625**
$(2D.68)_{16}$	2·16 + 13 + 6/16 + 8/256 = 32 + 13 + 0.375 + 0.03125	**45.40625**

Üçü eşittir ✓. Hatalı $(2D.0D)_{16}$ ise 45.05078125 verir ×.

Düşünme sorusu

00101.100 ve 101.1 aynı değeri taşır mı? Evet.

- 00101.100 = 0·16 + 0·8 + 4 + 0 + 1 + 0.5 + 0 + 0 = **5.5**
- 101.1 = 4 + 0 + 1 + 0.5 = **5.5**

Fark yalnızca tam kısmın solunda ve kesirli kısmın sağında olan, değeri değiştirmeyen sıfırlardır.

Değerlerin eşit olması bit sayılarının aynı olduğu anlamına gelir mi? Hayır. İlki 8 bit (5 + 3), ikincisi 4 bit (3 + 1) kullanır. Aynı değerin farklı uzunlukta gösterimleri olabilir. Bit sayısı, değeri değil, kullanılan **gösterim biçimini** (örneğin sabit 5.3 biçimi) belirtir.

YZ desteği ve doğrulama

Gruplama ve açıklamalar için Claude'dan yardım aldım. Bulunan $(55.32)_8$ ve $(2D.68)_{16}$ sonuçlarını, her gösterimi basamak ağırlıklarıyla onluya çevirerek kontrol ettim. Üçü de 45.40625 verdi.

Hatalı gruplamanın sonucu olan $(2D.0D)_{16}$'yı da aynı yolla hesapladım ve doğru değerden farkının 0.35546875 olduğunu gördüm.

Deney Yaprağı 4 – Ek: Kesirli Dönüşümde Kesinlik ve Yaklaşım Hatası

İncelenecek sayılar: 0.625, 0.1, 0.65, 0.975. Kesirli kısım için 8 ikili basamak kullanılacak. Son basamağın ağırlığı $2^{-8} = 1/256 = 0.00390625$'tir.

1. İkiliye dönüşüm (ardışık ×2, en çok 8 basamak)

Her adımda kesir 2 ile çarpılır. Tam kısım bir sonraki bittir, kesir bir sonraki adıma geçer.

0.625

Adım	1	2	3
×2	1.25	0.5	1.0
Bit	1	0	1

3. adımda kalan 0 olur ve dönüşüm **tamamlanır**: 0.101. Sağdan sıfırla 8 basamağa tamamlanırsa **0.10100000**.

0.1

Adım	1	2	3	4	5	6	7	8
×2	0.2	0.4	0.8	1.6	1.2	0.4	0.8	1.6
Bit	0	0	0	1	1	0	0	1

Kalan 0.6 ≠ 0, **tamamlanmadı → 0.00011001** (kesildi). Sonraki bitler 1, 0, 0, 1, … diye sürer.

0.65

Adım	1	2	3	4	5	6	7	8
×2	1.3	0.6	1.2	0.4	0.8	1.6	1.2	0.4
Bit	1	0	1	0	0	1	1	0

Kalan 0.4 ≠ 0, **tamamlanmadı → 0.10100110** (kesildi). Sonraki bitler 0, 1, 1, … diye sürer.

0.975

Adım	1	2	3	4	5	6	7	8
×2	1.95	1.9	1.8	1.6	1.2	0.4	0.8	1.6
Bit	1	1	1	1	1	0	0	1

Kalan 0.6 ≠ 0, **tamamlanmadı → 0.11111001** (kesildi). Sonraki bitler 1, 0, 0, 1, … diye sürer.

2. Kesilmiş gösterimlerin onluk değeri ve mutlak hatası

8 bitlik bir kesrin değeri (tam sayı karşılığı) / 256'dır.

Sayı	Kesik ikili	Değer	Mutlak hata
0.625	0.10100000	160/256 = 0.625	**0**
0.1	0.00011001	25/256 = 0.09765625	**0.00234375**
0.65	0.10100110	166/256 = 0.6484375	**0.0015625**
0.975	0.11111001	249/256 = 0.97265625	**0.00234375**

Kesme hatası her zaman $2^{-8} = 0.00390625$'ten küçüktür ✓.

3. En yakın 8 basamaklı değere yuvarlama (eşitlikte büyük olan seçilir)

Yöntem: sayı × 256 hesaplanır ve en yakın tam sayıya yuvarlanır. Bu, 9. bitin 1 mi 0 mı olduğuna bakmakla aynıdır.

Sayı	×256	Yuvarlanmış	İkili	Değer	Mutlak hata
0.625	160	160	0.10100000	0.625	**0**
0.1	25.6	26 (yukarı)	0.00011010	0.1015625	**0.0015625**
0.65	166.4	166 (aşağı)	0.10100110	0.6484375	**0.0015625**
0.975	249.6	250 (yukarı)	0.11111010	0.9765625	**0.0015625**

Yuvarlama hatası her zaman en fazla $\frac{1}{2} \cdot 2^{-8} = 0.001953125$'tir ✓. Hiçbir sayı tam orta noktada olmadığı için eşitlik kuralı bu sayılarda devreye girmedi.

4. Karşılaştırma tablosu ve = / ≈ kullanımı

Sayı	Kesik	Hata	Yuvarlanmış	Hata	Doğru yazım
0.625	0.10100000	0	0.10100000	0	$(0.625)_{10} = (0.101)_2$
0.1	0.00011001	0.00234375	0.00011010	0.0015625	$(0.1)_{10} \approx (0.00011010)_2$
0.65	0.10100110	0.0015625	0.10100110	0.0015625	$(0.65)_{10} \approx (0.10100110)_2$
0.975	0.11111001	0.00234375	0.11111010	0.0015625	$(0.975)_{10} \approx (0.11111010)_2$

- **=** yalnızca 0.625 için kullanılır, çünkü dönüşüm sonludur.
- Diğer üçü için **≈** kullanılmalıdır, çünkü hangi yöntem seçilirse seçilsin hata sıfır değildir.
- 0.65'te 9. bit 0 olduğundan kesme ve yuvarlama aynı sonucu verir. 0.1 ve 0.975'te 9. bit 1 olduğundan yuvarlama yukarı gider ve daha küçük hata verir.

5. Sonlu ikili kesir = tam sayı / 2^k

k basamaklı sonlu bir ikili kesir:

$0.b_1b_2\ldots b_k = b_1 \cdot 2^{-1} + b_2 \cdot 2^{-2} + \ldots + b_k \cdot 2^{-k} = (b_1 \cdot 2^{k-1} + b_2 \cdot 2^{k-2} + \ldots + b_k) / 2^k = \mathbf{m / 2^k}$

Payda her zaman 2'nin bir kuvvetidir. Örnek: $0.101_2 = 101_2 / 2^3 = 5/8$.

Sonuç: Sadeleştirilmiş p/q kesrinin sonlu ikili gösterimi **ancak ve ancak q, 2'nin bir kuvveti ise** vardır.

Sayı	Sade kesir	Payda	Sonlu mu?
0.625	5/8	2^3	Evet
0.1	1/10	$2 \cdot 5$	Hayır (5 çarpanı)
0.65	13/20	$2^2 \cdot 5$	Hayır (5 çarpanı)
0.975	39/40	$2^3 \cdot 5$	Hayır (5 çarpanı)

Düşünme sorusu

8 ikili kesir basamağı ile 8 onaltılı kesir basamağı aynı hassasiyeti sağlar mı? Hayır.

- 8 ikili basamakta son basamağın ağırlığı $2^{-8} = 1/256 \approx 0.0039$'dur.
- 8 onaltılı basamakta son basamağın ağırlığı $16^{-8} = 2^{-32} \approx 0.00000000023$'tür.

Her onaltılı basamak 4 bite karşılık gelir. Bu yüzden 8 onaltılı basamak 32 ikili basamağa eşdeğerdir ve $2^{24} \approx 16.7$ milyon kat daha hassastır. Hassasiyeti basamak sayısı değil, **son basamağın ağırlığı** belirler.

YZ desteği ve doğrulama

Dönüşüm tabloları için Claude'dan yardım aldım. Her kesik ve yuvarlanmış değeri (tam sayı)/256 olarak yeniden hesapladım. Kesme hatalarının 2^{-8}'den, yuvarlama hatalarının 2^{-9}'dan küçük olduğunu kontrol ettim.

Hangi sayıların sonlu ikili gösterime sahip olduğunu, sade kesirlerinin paydasının 2'nin kuvveti olup olmadığına bakarak ayrıca doğruladım.

Deney Yaprağı 5 – Ek: Bir Çözümdeki Hatanın Kaynağını Bulma

Denetlenecek rapor:

İddia	Rapordaki ifade
A	$(0.65)_{10} = (0.101)_{2}$
B	$(0.975)_{10} = (0.F99)_{16}$
C	$(101101.01101)_{2} = (2D.0D)_{16}$
D	Bir ikili tam sayının sağına sıfır eklemek değerini değiştirmez.

1. İddiaların değerlendirilmesi

İddia	Karar	Kısa gerekçe
A	**Yanlış (= yerine ≈ olmalı)**	0.65 = 13/20, payda 2'nin kuvveti değil. Sonlu ikili gösterimi yok, 0.101 sadece 3 bitte kesilmiş hali.
B	**Yanlış (= yerine ≈ olmalı)**	0.975 = 39/40, payda 2'nin kuvveti değil. Onaltılı açılım 0.F999… diye sonsuz sürer.
C	**Yanlış (gruplama hatası)**	Kesir solda sıfırla tamamlanmış. Doğrusu 2D.68 ve bu **tam eşitliktir**.
D	**Yanlış**	Sağa sıfır eklemek tam sayıyı 2 ile çarpar.

Tam eşitlik ile yaklaşık ifadenin ayrımı: Bir dönüşümün tam eşitlik (=) olarak yazılabilmesi için sağ taraftaki gösterimin sol taraftaki değeri **hiç hata olmadan** vermesi gerekir. Bunu belirleyen, sayının sade kesir hâlindeki paydasıdır: hedef taban 2, 8 veya 16 ise payda 2'nin bir kuvveti olmalıdır.

- **Yalnızca yaklaşık değer veren ifadeler: A ve B.** 0.65 = 13/20 ve 0.975 = 39/40 paydalarında 5 çarpanı taşır. Bu yüzden ikili ya da onaltılı gösterimleri sonsuzdur ve kaç basamak yazılırsa yazılsın sağ taraf sol tarafa eşit olmaz. Bu ifadelerde "=" değil "≈" kullanılmalıdır.
- **Tam eşitlik olabilen ifade: C.** Sol taraf zaten ikili bir sayıdır: 101101.01101_{2} = 1453/32, payda 2^{5}. Onaltılıya dönüşüm sonludur, yani $(2D.68)_{16}$ ile **tam eşittir**. C'deki sorun yaklaşıklık değil, hatalı gruplamadır. Düzeltildiğinde "=" doğru kullanılmış olur.
- D bir sayı dönüşümü değil, genel bir kural iddiasıdır ve yanlıştır (bkz. 4. madde).

2. A ve B: gerçek değerler ve mutlak hatalar

A: $(0.101)_{2} = 1\cdot½ + 0\cdot¼ + 1\cdot⅛ = 0.5 + 0.125 =$ **0.625** Mutlak hata = $|0.65 - 0.625| =$ **0.025**

Doğru açılım: 0.65 → ×2 adımları 1.3, 0.6, 1.2, 0.4, 0.8, 1.6, 1.2, 0.4 … → 0.10100110 0110… Rapor yalnızca ilk 3 biti almıştır.

B: $(0.F99)_{16} = 15/16 + 9/256 + 9/4096 = 0.9375 + 0.03515625 + 0.002197265625 =$ **0.974853515625** Mutlak hata = $|0.975 - 0.974853515625| =$ **0.000146484375**

Doğru açılım: 0.975 → ×16 adımları 15.6 (F), 9.6 (9), 9.6 (9), 9.6 (9) … → 0.F999… Rapor 3. basamakta kesmiştir.

B'nin hatası A'nınkinden yaklaşık 170 kat küçüktür, ama yine sıfır değildir. Bu yüzden "=" kullanılamaz.

3. C: hatalı sıfır tamamlamanın yeri

Kesirli kısım 01101 (5 bit), 4'lü gruplar için 8 bite tamamlanmalıdır.

- **Raporda yapılan (yanlış):** sıfırlar **noktanın yanına (sola)** eklenmiş: .**000**01101 → 0000 | 1101 → .0D
- **Doğrusu:** sıfırlar **sağa** eklenir: .01101**000** → 0110 | 1000 → .68

Tam kısım doğrudur: **00**101101 → 0010 | 1101 → 2D.

Doğru sonuç: $(101101.01101)_2 = (2D.68)_{16}$

Kontrol:

- İkili: 32 + 8 + 4 + 1 + 0.25 + 0.125 + 0.03125 = 45.40625
- $2D.68_{16} = 32 + 13 + 0.375 + 0.03125 = 45.40625$ ✓
- Raporun bulduğu $2D.0D_{16} = 45 + 13/256 = 45.05078125$ × (fark 0.35546875)

4. D: karşı örnek ve genel kural

Karşı örnek: $(101)_2 = 4 + 1 =$ **5**. Sağa bir sıfır eklenince $(1010)_2 = 8 + 2 =$ **10**.

Genel etki: Tam sayının sağına bir sıfır eklemek her biti bir basamak sola kaydırır, yani her bitin ağırlığını 2 katına çıkarır:

$\Sigma\, d_i \cdot 2^i \rightarrow \Sigma\, d_i \cdot 2^{i+1} = 2 \cdot (\Sigma\, d_i \cdot 2^i)$

k sıfır eklemek değeri 2^k ile çarpar ($101 \rightarrow 10100 = 20 = 5 \cdot 2^2$). Değerin değişmediği tek durum sayının 0 olmasıdır.

Değeri **değiştirmeyen** sıfır eklemeler şunlardır: tam kısmın **soluna** veya kesirli kısmın **sağına** (noktadan sonra) eklenen sıfırlar.

5. Düzeltilmiş ifadeler

İddia	Doğru ifade
A	$(0.65)_{10} \approx (0.101)_2$ (3 bitte kesilmiş, hata 0.025). Daha iyi: $(0.65)_{10} \approx (0.10100110)_2$
B	$(0.975)_{10} \approx (0.F99)_{16}$ (3 basamakta kesilmiş, hata ≈ 0.000146). Tam açılım $(0.F999\ldots)_{16}$
C	$(101101.01101)_2 = (2D.68)_{16}$
D	Bir ikili tam sayının sağına k sıfır eklemek değeri 2^k ile çarpar. Değeri koruyan, tam kısmın soluna veya kesirli kısmın sağına sıfır eklemektir.

Düşünme sorusu

İşlem adımları doğru görünürken sonuç ifadesi yanlış olabilir mi? Evet.

A'da her çarpma adımı doğrudur: 0.65×2 = 1.3 → 1, 0.3×2 = 0.6 → 0, 0.6×2 = 1.2 → 1. Bulunan ilk üç bit (101) gerçekten 0.65'in ilk üç bitidir. Hata hesapta değil, **sonucun ifade edilişindedir**: işlem kesilmiştir (kalan 0.2 ≠ 0), ama sonuç "=" ile tam eşitlikmiş gibi yazılmıştır. B için de aynı durum geçerlidir. Doğru adımlar, kesinlik bilgisi (= / ≈) ve hata sınırı belirtilmezse yanlış bir sonuç ifadesine dönüşebilir.

YZ desteği ve doğrulama

Bu yaprakta Claude'dan yardım aldım. 1. maddedeki "tam eşitlik olan ifade" kısmı ilk başta belirsizdi. Ders notlarındaki dönüşüm örneklerine bakarak yorumu netleştirdim: A ve B yalnızca yaklaşık değer veriyor, C ise düzeltildiğinde tam eşitlik oluyor.

A ve B'nin sağ taraflarını basamak ağırlıklarıyla onluya çevirerek mutlak hataları (0.025 ve 0.000146484375) kontrol ettim. C'nin doğru sonucunu iki ayrı yöntemle (onluya çevirme ve gruplama) buldum.

Deney Yaprağı 6 – Ek: Taban Bilgisi Kaybolmuş Kayıtların Çözümlenmesi

Problem: Üç cihaz aynı negatif olmayan tam sayıyı kaydetmiştir. Her cihaz 2, 8, 10 veya 16 tabanlarından birini kullanmıştır (tabanlar birbirinden farklı olabilir). Kayıtlar: A = 17, B = 21, C = 10001.

1. Geçerli kaynak tabanları

Bir kayıt, ancak bütün rakamları tabandan küçükse o tabanda geçerlidir. Yani en büyük rakam < taban olmalıdır.

Cihaz	Kayıt	En büyük rakam	Geçerli tabanlar	Geçersiz tabanlar
A	17	7	8, 10, 16	2 (7 ≥ 2)
B	21	2	8, 10, 16	2 (2 ≥ 2)
C	10001	1	2, 8, 10, 16	yok

2. Geçerli yorumların onluk karşılıkları

Kayıt	Taban 2	Taban 8	Taban 10	Taban 16
A = 17	geçersiz	$1\cdot8 + 7 = \mathbf{15}$	**17**	$1\cdot16 + 7 = \mathbf{23}$
B = 21	geçersiz	$2\cdot8 + 1 = \mathbf{17}$	**21**	$2\cdot16 + 1 = \mathbf{33}$
C = 10001	$2^4 + 1 = \mathbf{17}$	$8^4 + 1 = \mathbf{4097}$	**10001**	$16^4 + 1 = \mathbf{65537}$

3. Ortak değer ve tabanlar

Üç satırda da bulunan tek değer **17**’dir:

- $A = (17)_{10} = 17 \rightarrow$ **A cihazı 10 tabanı**
- $B = (21)_8 = 2\cdot8 + 1 = 17 \rightarrow$ **B cihazı 8 tabanı**
- $C = (10001)_2 = 16 + 1 = 17 \rightarrow$ **C cihazı 2 tabanı**

Ortak değer: 17_{10} (onaltılıda 11_{16}).

4. Başka çözüm var mı?

Değer kümeleri şunlardır:

- A: {15, 17, 23}
- B: {17, 21, 33}
- C: {17, 4097, 10001, 65537}

Kesişim: A ∩ B = {17}, {17} ∩ C = {17}.

A ile B’nin başka ortak değeri yoktur (15, 23 B’de; 21, 33 A’da yok). C’nin 17 dışındaki değerleri (4097, 10001, 65537) ise A ve B’nin hiçbir değerine eşit değildir. Bütün 3 × 3 × 4 = 36 taban birleşimi içinde yalnızca (10, 8, 2) birleşimi üç eşit değer verir. **Çözüm tektir.**

5. Üç kayıt da “10” olsaydı

Bu durumda A = 10, B = 10, C = 10’dur ve problemin koşulu aynıdır: üç cihaz aynı sayıyı kaydetmiştir. “10” kaydı dört tabanda da geçerlidir ve yorumları şunlardır:

Taban	2	8	10	16
“10” kaydının değeri	2	8	10	16

Her yorum farklı bir değer verir. Üç kaydın aynı değeri göstermesi için üç cihazın da **aynı tabanı** kullanması gerekir. Bu durumda dört geçerli çözüm vardır: hepsi 2 tabanı (değer 2), hepsi 8 (değer 8), hepsi 10 (değer 10) veya hepsi 16 (değer 16).

Tabanlar tek şekilde belirlenemez. Verilen kayıtlarda 17 tekti, çünkü kayıtların rakamları farklıydı ve farklı tabanlar aynı değere ancak tek bir yolla ulaşabiliyordu.

6. Belirsizliği önleyen kayıt biçimi

Önerilen biçim: **`<taban>#<rakamlar>`**. Taban her zaman onluk yazılır ve rakamlardan `#` ile ayrılır. Kaydı okuyan program önce tabanı (2, 8, 10, 16 olmalı), sonra her rakamın tabandan küçük olup olmadığını denetler.

Örnek kayıtlar (aynı değer, 17):

- `10#17`, A cihazı
- `8#21`, B cihazı
- `2#10001`, C cihazı

Alternatif olarak programlama dillerindeki önekler de kullanılabilir: `0b10001`, `0o21`, `17`, `0x11`. Ancak onluk için önek olmaması yine belirsizlik yaratabileceğinden `taban#` biçimi daha açıktır.

Düşünme sorusu

Bir kaydın bütün rakamlarının bir tabanda geçerli olması, kaydın o tabanda yazıldığını kanıtlar mı? Hayır.

Rakam geçerliliği **gerekli** ama **yeterli olmayan** bir koşuldur. Sadece hangi tabanların **olanaksız** olduğunu söyler. Örneğin C = 10001 dört tabanda da geçerlidir, ama gerçekte yalnızca 2 tabanında yazılmıştır ve bunu ancak diğer kayıtlarla karşılaştırarak bulabildik. Benzer şekilde “10” dört tabanda da geçerlidir ve hangisinde yazıldığı rakamlardan anlaşılamaz. Bir rakamın geçersiz olması bir tabanı kesin olarak eler, geçerli olması ise o tabanı yalnızca **aday** yapar.

YZ desteği ve doğrulama

Bu yaprakta Claude’dan yardım aldım. Ortak değerin 17 olduğunu ve başka çözüm bulunmadığını, 36 taban birleşiminin hepsini tabloda tek tek karşılaştırarak kontrol ettim.

5. maddeyi soruda yazıldığı gibi, üç kaydın da “10” olduğu durum olarak yorumladım.

Deney Yaprağı 7 – Ek: Tek Bit Değişiminin Sayısal Etkisi

Problem: $(173)_{10}$ değeri 8 bitlik işaretsiz ikili olarak tutuluyor. Her denemede başlangıç değerinin yalnızca bir biti tersine dönüyor. Geçerli aralık 0–200.

1. Başlangıç değeri

173 = 128 + 32 + 8 + 4 + 1

Bit konumu	7 (MSB)	6	5	4	3	2	1	0 (LSB)
Ağırlık	128	64	32	16	8	4	2	1
Bit	**1**	0	1	0	1	1	0	**1**

173_{10} = 10101101_2 (MSB = bit 7 = 1, LSB = bit 0 = 1)

2–3. Sekiz tek-bit hatası

Kural: Bit k, **1 → 0** dönerse değer 2^k **azalır**. **0 → 1** dönerse değer 2^k **artar**.

Deneme	Ters çevrilen bit	Ağırlık	Değişim	Hatalı ikili	Hatalı onluk	Yön	Mutlak hata
1	0 (LSB)	1	1→0	1010110**0**	172	azalma	1
2	1	2	0→1	1010111**1**	175	artma	2
3	2	4	1→0	1010**0**001	169	azalma	4
4	3	8	1→0	101**0**0101	165	azalma	8
5	4	16	0→1	101**1**1101	189	artma	16
6	5	32	1→0	10**0**01101	141	azalma	32
7	6	64	0→1	1**1**101101	237	artma	64
8	7 (MSB)	128	1→0	**0**0101101	45	azalma	128

Kontrol örneği (deneme 8): 00101101 = 32 + 8 + 4 + 1 = 45 = 173 − 128 ✓

4. “MSB’deki hata her zaman sayıyı artırır” iddiası

Yanlış. 173’te MSB zaten **1**’dir. Ters çevrilince 0 olur ve değer 128 **azalarak** 45’e düşer (deneme 8).

Bir bit hatası, bitin önceki değerine göre sayıyı artırır ya da azaltır. MSB ancak 0 iken (yani sayı < 128 iken) ters çevrilirse sayı artar. Doğru ifade: “MSB’deki hata sayıyı her zaman **en büyük miktarda** (128) değiştirir.”

5. 0–200 aralık kontrolü

Deneme	Hatalı değer	Aralıkta mı?	Kontrol yakalar mı?
1	172	evet	hayır
2	175	evet	hayır
3	169	evet	hayır
4	165	evet	hayır
5	189	evet	hayır
6	141	evet	hayır
7	**237**	**hayır (> 200)**	**evet**
8	45	evet	hayır

Aralık kontrolü 8 hatadan **yalnızca birini** (bit 6, 237) yakalar.

Aralıkta kalan hatalı değer örneği: Deneme 8 → **45**. Bu, en büyük hatadır (128), ama 0–200 aralığında olduğu için kontrolden geçer. Aralık kontrolü yalnızca değeri imkânsız kılan hataları yakalayabilir. Olası ama yanlış değerleri yakalayamaz. Bunun için eşlik biti (parity) gibi ek denetimler gerekir: tek bit hatalarının hepsi bitlerdeki 1 sayısının tek/çift olma durumunu değiştirir.

6. Onaltılı karşılaştırma

173_{10} = 1010 | 1101 = $\mathbf{AD_{16}}$

Deneme	Bit	Hatalı ikili	Onaltılı	Değişen hane
–	–	1010 1101	**AD**	–
1	0	1010 1100	A**C**	düşük hane
2	1	1010 1111	A**F**	düşük hane
3	2	1010 1001	A**9**	düşük hane
4	3	1010 0101	A**5**	düşük hane
5	4	1011 1101	**B**D	yüksek hane
6	5	1000 1101	**8**D	yüksek hane
7	6	1110 1101	**E**D	yüksek hane
8	7	0010 1101	**2**D	yüksek hane

Gözlem: Her onaltılı hane tam 4 bite karşılık gelir. Bitler 0–3'teki hatalar yalnızca düşük haneyi (D), bitler 4–7'deki hatalar yalnızca yüksek haneyi (A) değiştirir. Bu yüzden onaltılı gösterimde hatanın **hangi 4 bitlik grupta** olduğu kolayca görülür.

Düşünme sorusu

"Tek bir bit değiştiği için sayısal hata küçüktür" ifadesi neden güvenilir değildir?

Çünkü hatanın büyüklüğü değişen bit **sayısına** değil, değişen bitin **konumuna** bağlıdır. Konumu k olan bitin ağırlığı 2^k'dir:

$|\text{hata}| = 2^k$

Yukarıdaki tabloda tek bit hatası 1 (LSB) ile 128 (MSB) arasında değişmiştir. MSB hatası değeri 173’ten 45’e, yani %74 oranında düşürmüştür. Her bir konum yukarı çıkıldığında hata 2 katına çıkar. n bitlik bir sayıda en büyük tek-bit hatası 2^{n-1}’dir, yani bütün aralığın yarısıdır.

YZ desteği ve doğrulama

Tablo ve açıklamalar için Claude’dan yardım aldım. Her hatalı değeri iki ayrı yoldan hesapladım: ikili gösterimi basamak ağırlıklarıyla toplayarak ve 173 ± 2^k formülüyle. İki yol her denemede aynı sonucu verdi.

Onaltılı karşılıkları da 4’lü bit gruplarıyla ayrıca kontrol ettim.